Learning Objectives

This lab lets you practice writing programs using dictionaries.

Estimated Size

1 program file: `dictionaries.py`, 3 functions, and less than 15 lines of code in each function.

Setup

In your Python files, you must include author information. On the first line, add a `# Author` comment line, where you mention your full name. Similarly, include your email and Spire ID on the `# Email` and `# Spire ID` comment lines, respectively. Remember, these comments must appear **at the beginning of every Python file** that you submit to Gradescope. For example:

```
# Author   : John Snow
# Email    : jsnow@umass.edu
# Spire ID : 12345678
```

1. Implement function `count_words` (5 points)

In lecture 13, you have learned Python's dictionary data type. A dictionary can contain any number of key-value pairs (i.e. entries), and it is used to represent associate relationships. You can create a dictionary using curly brackets `{}`, or using the `dict()` function.

A dictionary is a very versatile data type for many applications. Counting the frequency of words is a classic example. Let's say we want to find out the frequencies of words that appear in a textbook, or in Internet search terms. The input is a tuple that contains many individual words. Our goal is to create a dictionary that stores each unique word, and the number of times each word has occurred. So for each entry, the key is a string (a word) and the value is an integer (the number of occurrences).

Create a new file in VSCode and save/name it `dictionaries.py`. Your first exercise is to write a function called `count_words` which takes a tuple of words as input, and **returns a dictionary** where the key of each entry is a unique word from the input, and the value is the number of times that word appears in the input. Specifically, your function should:

- Take **a tuple of strings** as the input parameter. You may assume **the strings are all lower-case**.
- Create an empty dictionary to begin with, for example by `some_dict = {}`
- Loop over the input tuple. For each string in the tuple, if it doesn't exist in the dictionary yet (you may use the `in` operator to check if a key exists in the dictionary), add it to the dictionary with a value of `1` (i.e. first occurrence). If it already exists, increment the value by `1`.
- After the loop, **return the dictionary**.
- Do **NOT** ask the user for any input. Do **NOT** print anything in your function.

When you are done, write some test cases yourself to check if the return value of your function is as expected. For example:

```
words = ('he', 'saw', 'a', 'saw', 'saw', 'a', 'saw')
print(count_words(words))
```

```
Output → {'he': 1, 'saw': 4, 'a': 2}
```

Because the keys in the dictionary are unordered, your output may look different from the example above. But it should contain the same set of unique words, and the count of each word should match the reference solution above.

2. Implement function `average_prices` (5 points)

Every month the Bureau of Labor Statistics calculates the Consumer Price Index (CPI) to understand the rate of inflation. They would collect the prices of individual commodities from a large number of stores all over the country, and calculate the **average price** of each unique commodity. The result is then compared with the previous month to estimate the rate of inflation.

In this exercise, you will write a function called `average_prices` that takes a collection of commodities and their prices, and returns a dictionary where the key of each entry is a unique commodity name from the input, and the value is the average price of that commodity. The input is given as a tuple, where each item is itself a 2-element tuple representing the name of the commodity and its price. For example: `(('gouda cheese 1 lbs', 3.49), ('organic oyster mushroom 1 lbs', 6.89), ('toilet paper 1 roll', 3.99), ('apple juice 1 gallon', 7.99), ('gouda cheese 1 lbs', 4.29), ('toilet paper 1 roll', 4.19), ('talenti gelato vanilla', 5.59))`

As you can see, it's a tuple of tuples: the size of the top-level tuple is the number of collected commodities, and each item in the top-level tuple is a 2-element tuple of the commodity name and price. Specifically, your function should:

- Take the input described above as the parameter. Assume **all commodity names are lower-case**.
- Return a **dictionary** where the key of each entry is a unique commodity name (a string) from the input, and the value is the **average price** of that unique commodity.
- Do **NOT** ask the user for any input. Do **NOT** print anything in your function. Do **NOT** use nested loops as this exercise does NOT need nested loops. (You may use other functions in the loop however, like **sum()** or **len()**)

Hint: there are several ways to implement this function. For example, you may create two dictionaries one storing the **total price**, and the other the **total number** of every unique commodity seen so far as you loop over the input. Then in the end you use the two dictionaries to calculate the **average price** of each unique commodity.
Remember: If you loop over a dictionary, you are only accessing the keys, and must use those keys to access the values.

Alternatively, you may create one dictionary where the key of each entry is the name of a unique commodity, and the value is a **list** of the prices of that commodity. As you loop over the input, you can append a price to the list of its corresponding commodity in the dictionary. In the end you use this dictionary to calculate the average price of each unique commodity.

Below is an example input and its expected return value (for simplicity, we use single letters to represent commodity names).

```
prices = (('a', 1.0), ('c', 4.2), ('b', 3.9), ('a', 1.2), ('d', 10.4), ('b', 4.3), ('b', 3.8))
print(average_prices(prices))
```

```
Output → {'a': 1.1, 'c': 4.2, 'b': 4.0, 'd': 10.4}
```

3. Implement function `count_bigrams` (5 points)

In Natural Language Processing, often we are not only interested in counting individual words, but we are interested in co-occurrent words. A **bigram** (or 2-gram) is a sequence of two adjacent words in a sentence. For example, the sentence `'I am taking a Python class'` has 6 individual words and 5 bigrams: `(I am)`, `(am taking)`, `(taking a)`, `(a Python)`, `(Python class)`. Given a large collection of texts, we can create a dictionary that contains all unique bigrams, and count how many times each bigram occurs. This allows us to analyze the frequency of words that co-occur one after another. For example, you may find that the bigram `(this is)` occurs frequently, so if you see the word `this`, there is a high probability that the next word following it would be `is`.

Write a function called `count_bigrams` which takes a tuple of individual words as input, and **returns a dictionary** where the key of each entry is a unique bigram from the input, and the value is the number of times that bigram occurs. In this exercise, we will represent a bigram by **a tuple of two words**, for example: `('this', 'is')`. Specifically, your function should:

- Take **a tuple of strings** as the input parameter. You may assume **the strings are all lower-case**.
- Create an empty dictionary to begin with, for example by `some_dict = {}`
- Loop over the input tuple. Each word and its following word form a bigram, which you should represent by **a tuple of the two words.** If a bigram does not exist in the dictionary yet, add it to the dictionary with a value of `1` (i.e. first occurrence). If it already exists, increment the value by `1`.
- After the loop, **return the dictionary**.
- Do **NOT** ask the user for any input. Do **NOT** print anything in your function. Do **NOT** use nested loops as this exercise does NOT need nested loops. Using a nested loop would mean we need to compare/use every string in comparison to every other string, this is not needed as bigrams are only concerned with the string directly after or before the current string.

When you are done, write some test cases to check if your function is correct. For example:

```
count_bigrams(())         # returns {}
count_bigrams(('hello',)) # returns {} (note the trailing comma, which indicates
                          # this is a 1-element tuple instead of just a string)
```

```
The above two both return an empty dictionary {} as there are fewer than 2 words
in the input, which are insufficient to make any bigram.
```

```
words = ('she', 'knows', 'and', 'she', 'knows', 'that', 'he', 'knows', 'that', 'she', 'knows')
```

```
print(count_bigrams(words))
```

```
Output → {('she', 'knows'): 3, ('knows', 'and'): 1, ('and', 'she'): 1, ('knows',
'that'): 2, ('that', 'he'): 1, ('he', 'knows'): 1, ('that', 'she'): 1}
```

Again, because the keys in the dictionary are unordered, the ordering of bigrams in your output may be different from the example above. But it should contain the same set of unique bigrams, and the count of each bigram should match the reference solution above.

4. Submission to Gradescope

Log in to Gradescope, select **Lab 07: (Code)**, and submit the required file. Wait until the autograder finishes running and returns the result to you. You can resubmit as many times as you like before the deadline.

Also remember to submit the lab questionnaire. If you see a question you do not know how to answer, ask TAs/UCAs in your lab, or use piazza, or go to one of our office hours.

(Common Paragraph) Academic Honesty

All work that is completed in this assignment is your own (if you work individually) or your own group's (if you work in a group). You may talk to other students about the problems, and brainstorm about solutions. However, you may NOT share code in any way, except with your group members. What you submit **must be your own or your own group's work**.

You may not use any code that is posted on the internet. If you are not sure it is in your best interest to contact the course staff. We will be using software that will compare your code to other students in the course as well as online resources. It is very easy for us to detect similar submissions and will result in a failure for the exercise or possibly a failure for the course. Please, do not do this. It is important to be academically honest and submit your work only. Please review the UMass Academic Honesty Policy and Procedures so you are aware of what this means.

Copying solutions, whether partial or whole, obtained from other students or elsewhere, is academic dishonesty. Do not share your code with your classmates, and do not use your classmates' code. If you are confused about what constitutes academic dishonesty you should re-read the course policies. We assume you have read the course policies in detail and by submitting this project you have provided your virtual signature in agreement with these policies.